
MindMap AI

Local AI Mind Map Builder

DAY 06 | JULY 3, 2026

Type a topic. The AI builds the map. Tap to go deeper.

Category	Details
Startup Name	MindMap AI
Type	Offline AI Mind Mapping Tool (Mobile-Only)
Stack	Vanilla HTML / JS + Canvas API + Ollama
Models	llama3.2, mistral, gemma2, qwen2.5
Privacy	100% Local — Zero Data Leaves Device
Author	Mayank Swaraj / Mayank Creations
Series	Daily Local LLM Startup — Day 06

WHITEPAPER

MindMap AI: On-Device Visual Knowledge Expansion

Abstract

MindMap AI is a mobile-first, offline mind-mapping tool that uses a locally-running LLM to auto-generate structured knowledge maps from any topic string. Users can tap any leaf node to AI-expand it into sub-concepts, export the full outline as plain text, and revisit past maps from a local history — all without internet connectivity.

1. Problem Statement

Existing mind-mapping tools (MindMeister, XMind, Miro) are cloud-dependent and require manual node creation. AI-assisted mind-mapping tools (Whimsical AI, Ayoa) route topic data through cloud APIs. Neither option works offline or guarantees that your research topic, business idea, or study content is not stored on a third-party server.

2. Solution

- Topic-to-map generation — one prompt produces a structured 5-branch, 3-leaf-per-branch map as JSON
- Canvas rendering — pure HTML5 Canvas draws the radial map with no SVG library or charting dependency
- Tap-to-expand — tapping any leaf fires an on-demand expansion prompt, adding 3 sub-concepts in place
- Outline tab — auto-generated hierarchical text outline mirrors the visual map at all times
- Text export — full outline exported as .txt file via Blob URL for sharing or note-taking
- History — last 20 maps stored in localStorage, one-tap reload

3. Technical Architecture

Layer	Technology	Role
UI	Vanilla HTML5/CSS3 (mobile-only)	3-tab layout: Visual, Outline, History. Max-width 480px
Rendering	HTML5 Canvas 2D API	Radial layout: root at center, branches at equal angles, leaves offset
Logic	Vanilla JS (ES6+)	JSON map parsing, node position storage, touch/click tap detection
Storage	localStorage	Up to 20 full map objects including expanded nodes
AI Runtime	Ollama local process	Generation (5 branches x 3 leaves) + per-node expansion (3 sub-concepts)
Layout Algo	Radial angle distribution	Branch angle = $(2 \cdot \pi / n) \cdot i - \pi / 2$; leaf spread = $\pi / 3$ across branch fan

4. Market Opportunity

The global mind-mapping software market exceeded USD 1.1 billion in 2024. Study tools, brainstorming aids, and knowledge management tools represent the dominant use cases. The offline-AI angle targets students, researchers, and content creators who frequently work in low-connectivity environments (libraries, flights, remote areas) and want AI assistance for ideation without cloud exposure of their research topics.

5. Monetization

Tier	Price	Offering
Free	USD 0	Unlimited maps, 20 history cap, 3 expansions/map
Pro	USD 5/mo	Unlimited expansions, PNG export, 200 history cap
Lifetime	USD 29	Everything in Pro forever
Education	USD 49/classroom/yr	Class-wide topic library, shared map templates

6. Competitive Advantage

Feature	MindMap AI	MindMeister	Whimsical AI	XMind
Offline AI Generation	YES	NO	NO	NO
Tap-to-Expand	YES	NO	Limited	NO
No Account Required	YES	NO	NO	Partial
Mobile-First Design	YES	Partial	NO	Partial
Single HTML File	YES	NO	NO	NO

02

SOURCE CODE PAPER

Annotated technical breakdown of MindMapAI.html

SOURCE CODE PAPER

MindMapAI.html — Annotated Technical Breakdown

Overview

MindMap AI is a single HTML file that uses the HTML5 Canvas 2D API for rendering — no SVG, no D3, no charting library. The radial layout algorithm is hand-written in ~30 lines of vanilla JS. Node positions are stored in a global array after each draw call, enabling accurate touch/click tap detection.

Section 1: Radial Layout Algorithm

Branches are distributed at equal angles around the root:

```
var angle = (2*Math.PI / n) * i - Math.PI/2;
var bx = cx + Math.cos(angle) * branchDist;
var by = cy + Math.sin(angle) * branchDist;
// Leaves fan around each branch:
var spread = (PI/3) * (j - (leaves.length-1)/2);
var leafAngle = angle + spread;
```

Subtracting $\text{PI}/2$ from the start angle rotates the first branch to the top (12 o'clock position) rather than the right, which is more natural for mind maps. The leaf fan spread of $\text{PI}/3$ (60 degrees total) prevents leaf collisions across most branch counts.

Section 2: Tap Detection on Canvas

After each `drawMap()` call, node positions are stored in `window._nodePos` with `x`, `y`, `r`, and type metadata. Tap detection iterates this array and checks Euclidean distance:

```
for(var i=0; i<_nodePos.length; i++){
var dx=cx-n.x, dy=cy-n.y;
if(Math.sqrt(dx*dx+dy*dy) <= n.r+4){
// node tapped -> expandLeaf()
}
}
```

The +4 pixel tolerance accounts for finger imprecision on small leaf nodes (radius 22px). Touch events use `e.touches[0]` coordinates offset by `canvas.getBoundingClientRect()`.

Section 3: Expand-in-Place Mutation

When a leaf is expanded, the string in `branches[brIdx].nodes[leafIdx]` is replaced with an object:

```
// Before expansion:
branches[0].nodes[1] = "Neural Networks"
// After tap expansion:
```

```
branches[0].nodes[1] = {
name: "Neural Networks",
expanded: ["Backpropagation","CNNs","Transformers"]
}
```

drawMapExpanded() detects object vs string nodes, draws the standard map first, then renders an additional sub-fan of 3 nodes around each expanded leaf. renderOutline() reads the same structure, outputting indented sub-points.

Section 4: Function Reference

Function	Role
generateMap()	Fires JSON-constrained prompt, parses 5-branch structure, calls drawMap() and renderOutline
drawMap()	Clears canvas, draws root + all branches + leaves via radial algo, stores node positions
drawNode(x,y,r,fill,label,color,fs,glow)	Draws circle + word-wrapped label at given coordinates
onTap(e)	Detects tapped node from _nodePos via Euclidean distance, delegates to expandLeaf()
expandLeaf(brIdx,leafIdx,name)	Fires 3-item expansion prompt, mutates branch node to object, redraws
drawMapExpanded()	Calls drawMap() then draws sub-fans around expanded leaf objects
renderOutline()	Converts currentMap to indented HTML list, handles expanded node objects
exportOutline()	Serializes currentMap to plain text, triggers Blob download

03

HISTORY PAPER

Origin story, design decisions, and build log

HISTORY PAPER

MindMap AI — Origin, Design Decisions & Build Log

Origin Story

MindMap AI is Day 6 of the Mayank Creations Daily Local LLM Startup series, built July 3, 2026. The series has covered text output tools (Days 1-3), vision input (Day 4), and code analysis (Day 5). Day 6 introduces a new dimension: visual spatial output rendered on canvas, making it the first tool in the series with a custom rendering engine rather than a DOM-based UI.

The motivating question: can a local LLM replace the manual effort of mind-mapping well enough that a user never needs to drag a single node? The answer, proven by this build, is yes — for ideation and study purposes at minimum.

Design Philosophy

Canvas over DOM

DOM-based mind maps require either an SVG library (D3, Raphael) or a force-layout engine (Cola.js). Both violate the zero-dependency constraint. A hand-written Canvas 2D renderer fits in 60 lines and produces a visually superior result for radial layouts.

Purple/Violet Palette

Purple (#c084fc) evokes creativity, ideation, and thought — appropriate for a brainstorming tool. It also maintains visual distinction from the series palette history: purple (Day 1 journaling) was a deliberate callback, repositioned here with a brighter, more energetic violet.

5 Branches x 3 Leaves as Default

This specific structure (15 total nodes) was chosen by testing: fewer branches produce sparse maps that look incomplete; more branches produce visual collisions at 480px width. 3 leaves per branch is the maximum that fans cleanly within a $\pi/3$ spread without overlap.

Expand on Tap

Rather than generating an arbitrarily deep tree upfront (which would overflow the canvas and take 30+ seconds on a local model), depth-on-demand keeps the initial generation fast (2-4 seconds) and lets the user explore only the branches they care about.

Build Log — July 3, 2026

Time	Action	Decision
T+0:00	Idea selection	Visual output tool — first Canvas-based build in the series

T+0:15	Layout algorithm	Radial with $\pi/2$ offset for top-start; $\pi/3$ leaf fan; tested at 480px width
T+0:40	JSON schema	5 branches, 3 leaves each — hardcoded counts ensure consistent layout
T+1:00	drawNode()	Word-wrap algorithm for canvas labels: measures text width, splits at $\text{maxW} = r * 1.6$
T+1:20	Tap detection	<code>_nodePos</code> global array; Euclidean distance check with +4px touch tolerance
T+1:40	Expand-in-place	Node mutation from string to object; <code>drawMapExpanded()</code> layered over <code>drawMap()</code>
T+2:00	Outline + export	Auto-sync outline tab; Blob URL text export for sharing
T+2:15	History	Full map JSON stored per entry including expanded node state
T+2:30	Documentation	Whitepaper, Source Code Paper, History Paper compiled into single PDF

Key Decisions Rejected

Rejected	Why Rejected
D3.js force layout	External dependency; D3 force graphs require physics simulation which is unpredictable on sm
SVG rendering	SVG DOM manipulation for 15+ nodes is verbose without a library; Canvas 2D is simpler for th
Editable node labels	Inline canvas text editing requires a floating input overlay with complex positioning; deferred to
Unlimited expansion depth	Deep trees overflow 480px canvas; depth-on-demand is both faster and more usable on mobile
PNG export	<code>Canvas.toBlob()</code> works but requires handling safe-area cropping and DPI scaling; deferred to P

What Day 7 Could Build On

- Pinch-to-zoom and pan on the canvas for larger maps
- PNG/SVG export of the rendered map
- Editable node labels via floating input overlay
- Auto-layout adjustment for expanded nodes to prevent overlap
- Curved bezier lines instead of straight lines between nodes
- Color-theme selector (Light / Dark / Pastel)